

Магистратура
«Искусственный интеллект»

Инструменты разработки
Лекция 6

Чернов Евгений
НИУ ВШЭ

Контейнеризация и виртуализация

Что такое контейнер на уровне ОС: namespaces и cgroups, и чем он отличается от виртуальной машины — ещё до Docker

План занятия

1. **Проблема**
   «works on my machine», изоляция и воспроизводимость
2. **Виртуализация**
   гипервизоры тип 1 и тип 2, полная эмуляция железа
3. **Контейнеризация**
   общее ядро и изоляция процессов
4. **Namespaces**
   pid, net, mnt, uts, ipc, user
5. **cgroups**
   лимиты CPU, памяти и ввода-вывода
6. **VM или контейнер**
   вес, скорость старта, когда что применять

«А у меня всё работает»

Код запускается у автора, но падает у коллеги или на сервере. Причина — различия в окружении: версии языка, библиотеки, системные пакеты, драйверы.

одна программа, разное окружение

```
# у меня на ноутбуке
$ python app.py
OK   (Python 3.11, CUDA 12)

# на сервере коллеги
$ python app.py
ModuleNotFoundError: torch
# Python 3.8, другой CUDA
```

Изоляция

у каждого приложения своё окружение, которое не мешает соседям на той же машине

Воспроизводимость

одинаковый запуск на ноутбуке, у коллеги и на сервере — «собрал раз, работает везде»

01

РАЗДЕЛ 01

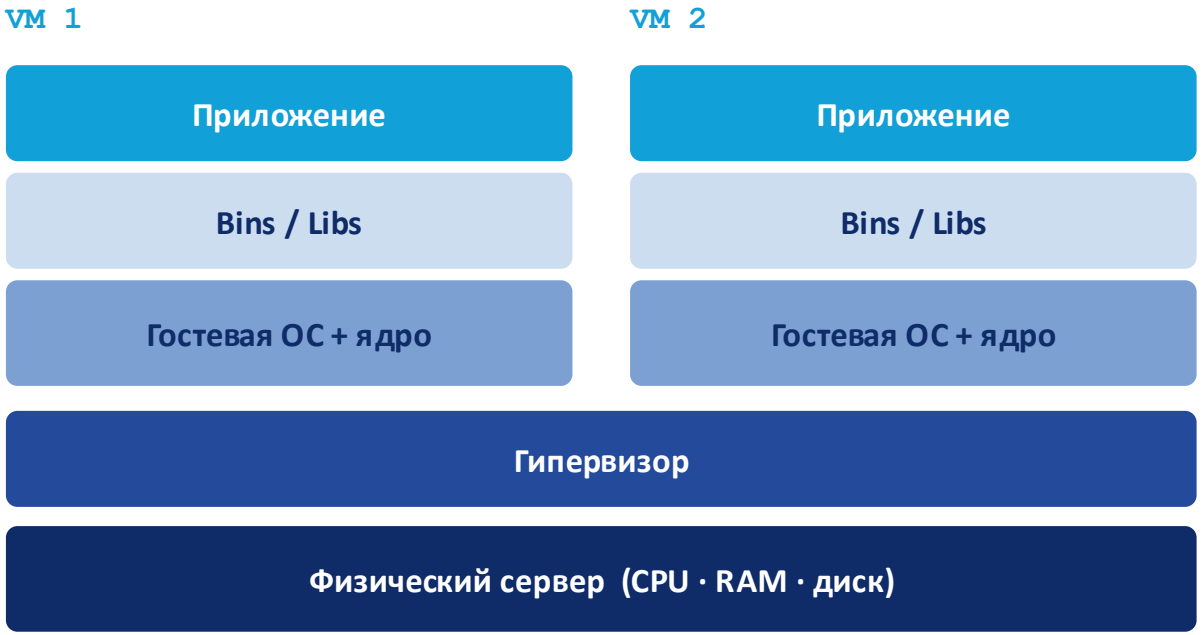
Две технологии изоляции

Виртуализация повторяет железо целиком; контейнеризация делит одно ядро

Гипервизор и полная виртуальная машина

Виртуализация эмулирует железо: у каждой ВМ — своя полноценная гостевая ОС и своё ядро.

на одном сервере — несколько изолированных ВМ



Тип 1 — bare-metal

ставится прямо на железо. ESXi, KVM, Hyper-V. Дата-центры и облака.

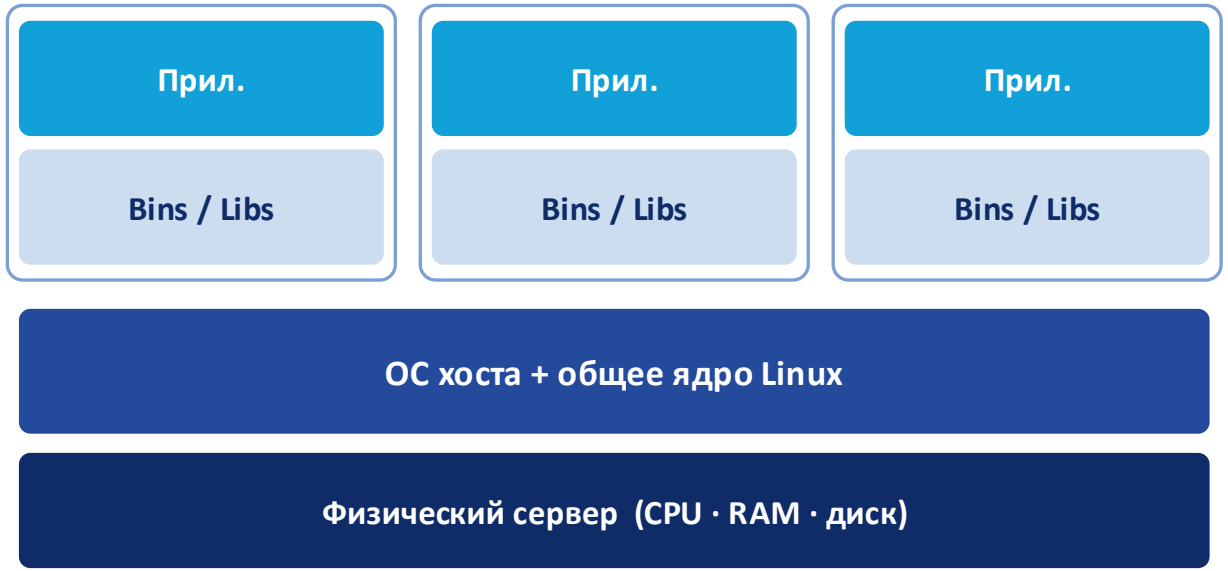
Тип 2 — hosted

программа поверх обычной ОС. VirtualBox, VMware Workstation. Рабочий ноутбук.

Общее ядро и изоляция процессов

Контейнер — это не мини-ВМ. Это обычный процесс хостовой ОС: все контейнеры делят одно ядро, но за счёт изоляции не видят друг друга.

контейнеры — процессы поверх общего ядра



Нет гостевой ОС

контейнер использует ядро хоста напрямую — своей ОС и своего ядра у него нет

Изоляция + лимиты

namespaces прячут чужие процессы, сеть и файлы; cgroups ограничивают ресурсы

Аналогия: многоквартирный дом

Пока без команд. Представьте дом: одно здание, много квартир. Так и контейнеры делят одно ядро, но каждый живёт изолированно.

Дом = одно ядро хоста (общее на всех)

Кв. 1	Кв. 2	Кв. 3
контейнер свой быт	**контейнер** свой быт	**контейнер** свой быт

Стены и двери `namespaces`

сосед не виден и не слышен: свой звонок, свой почтовый ящик, свой адрес

Счётчики ресурсов `cgroups`

лимит воды и света на квартиру — один жилец не осушит весь дом

! **Контейнер — как квартира:** своя, изолированная, с лимитами. Но дом — ядро — у всех жильцов один.

Namespaces: что процесс «видит»

Namespace изолирует один вид системных ресурсов: процесс внутри видит только свой срез системы. Ядро одно — но представлений много.

pid

своё дерево процессов; внутри PID 1 — ваш процесс

net

свои интерфейсы, IP-адреса, порты и маршруты

mnt

своя иерархия точек монтирования — файловая система

uts

своё имя хоста (hostname) и домен

ipc

своя очередь сообщений и разделяемая память

user

свои UID/GID; root внутри ≠ root снаружи

Один процесс — два взгляда

Namespace — это не копия системы, а точка зрения. Один и тот же процесс ядро показывает по-разному внутри контейнера и снаружи, на хосте.

Что смотрим	Снаружи — на хосте	Внутри namespace
PID процесса `pid`	40213 — обычный процесс	**1 — как будто init системы**
Имя хоста `uts`	laptop-hse	**container-demo**
Сеть `net`	eth0 + внешний IP	**только lo, и тот пустой**

cgroups: сколько ресурсов процесс «может съесть»

namespaces отвечают на вопрос «что видно». cgroups (control groups) — на вопрос «сколько можно»: CPU, память и ввод-вывод.

cpu.max
Процессор
доля CPU-времени, напр. 50% ядра

memory.max
Память
потолок RAM; превысил → OOM-kill

io.max
Диск (I/O)
лимит скорости чтения и записи

лимит процессора

```
# cgroup v2 — лимит CPU
# «половина одного ядра»
$ echo "50000 100000" > cpu.max
# формат: quota period (мкс)
# 50000/100000 = 50%
$ cat cpu.stat
throttled_usec 1240300
```

02

РАЗДЕЛ 02

VM или контейнер?

Одна задача — изоляция, но разная цена: вес, скорость старта и границы безопасности

Два стека рядом: тяжёлый и лёгкий

Одна цель — изолированное приложение. Слева виртуализация тащит полную ОС в каждую ВМ; справа контейнеры делят ядро хоста.

Виртуальные машины

тяжёлые: своя ОС в каждой



HV II-го типа

Контейнеры

лёгкие: общее ядро хоста

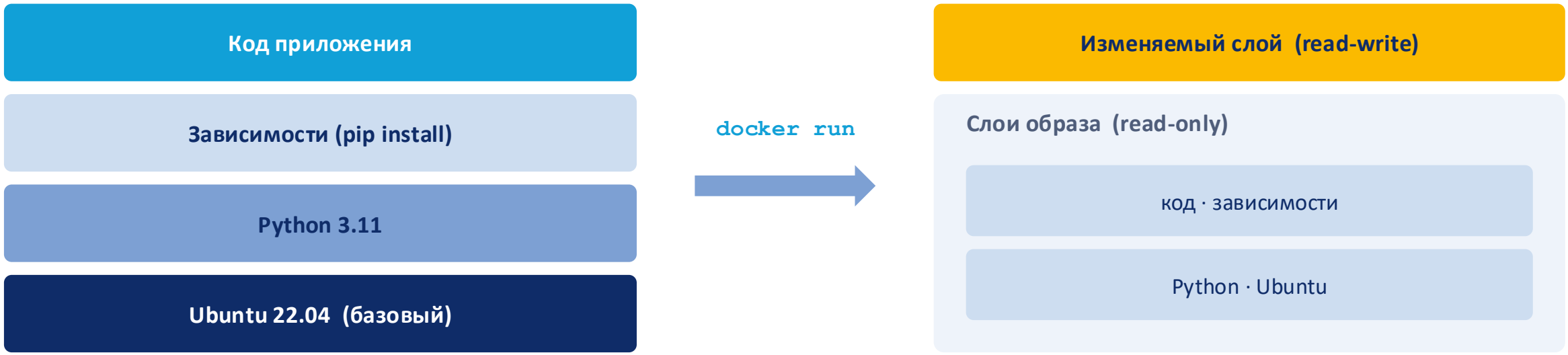


VM против контейнера: по пунктам

Критерий	Виртуальная машина	Контейнер
Изоляция	полная — своё ядро и железо	**уровень процесса, общее ядро**
Вес	гигабайты — ОС внутри	**мегабайты — только приложение**
Старт	десятки секунд (грузит ОС)	**миллисекунды (просто процесс)**
Применение	другая ОС, жёсткая изоляция	**микросервисы, CI/CD, масштаб**

Образ против контейнера: слои

Образ — шаблон «только для чтения» из слоёв. Контейнер — запущенный образ плюс тонкий изменяемый слой сверху. Один образ → много контейнеров, слои не копируются.



OverlayFS: слои складываются в один «/»

Как один образ обслуживает десятки контейнеров и при этом не копируется? Файловая система склеивает слои: слои образа — только для чтения, а свой тонкий слой для записи получает каждый контейнер.

На диске — слои

upperdir · r/w — правки контейнера

lowerdir · r/o — слои образа (общие)

код · зависимости

Python · Ubuntu

`overlay`

Что видит процесс

`единая ФС «/»`

один «/» из всех слоёв сразу; верхний слой перекрывает нижний

процесс не знает, что это слои

Copy-on-write: меняешь файл — ядро копирует его в r/w слой, образ не трогается. Удалил контейнер — правки исчезли (для данных нужны volumes).

Контейнер «руками»: unshare и PID namespace

Контейнер можно собрать без Docker. unshare запускает процесс в новых namespaces — покажем изоляцию дерева процессов вживую.

терминал

```
# новый PID + mount namespace
$ sudo unshare -p -f --mount-proc bash
# внутри — своё дерево процессов
$ ps -e
  PID TTY          TIME CMD
    1 pts        00:00 bash
    2 pts        00:00 ps
# наш bash стал PID 1
```

unshare
запускает процесс в новых namespaces

-p -f
новый PID-namespace + fork дочернего

--mount-proc
свой /proc → ps видит только своё

PID 1
первый процесс в namespace — ваш bash

Лимиты в действии: cgroup и OOM-killer

Ограничим ресурсы: создадим cgroup v2, зададим потолок памяти и посмотрим, как ядро убивает процесс при превышении.

терминал

```
$ CG=/sys/fs/cgroup/demo
$ mkdir $CG
# потолок памяти — 50 МБ
$ echo 50M > $CG/memory.max
# помещаем shell в группу
$ echo $$ > $CG/cgroup.procs
# просим гигабайт памяти:
$ python -c "x='a'*10**9"
Killed  (OOM в cgroup demo)
```

memory.max
жёсткий потолок памяти для группы

cgroup.procs
какие процессы входят в cgroup

OOM-killer
превысил лимит → ядро убивает процесс

хост цел
остальная система работает как обычно

Когда виртуальная машина, когда контейнер

Это не «или-или»: часто контейнеры запускают внутри облачной VM. Но для типичных задач выбор понятен.

Виртуальная машина

- нужна другая ОС (Windows на Linux)
- жёсткая изоляция и безопасность
- эмуляция окружения целиком, своё ядро
- долгоживущая «машина» с полным доступом

Контейнер

- упаковать приложение и зависимости
- быстрый старт и высокая плотность
- микросервисы, CI/CD, масштабирование
- одинаковый запуск dev → prod

! **На практике их сочетают:** облачная VM как хост, а приложения внутри неё — в контейнерах.

03

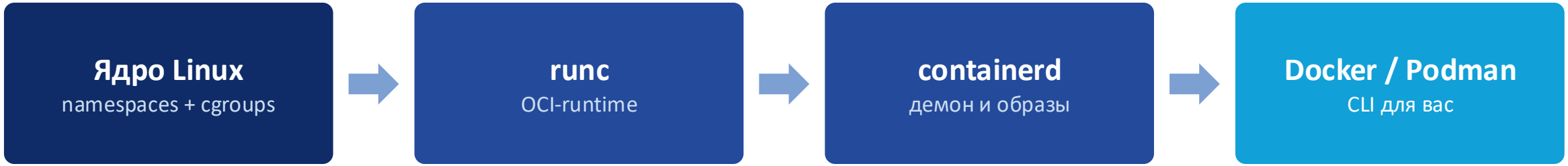
РАЗДЕЛ 03

Docker и сказки о контейнерах

Те же namespaces и cgroups, однако, через "docker run"

Docker — обёртка над ядром

Docker не изобретает изоляцию. Под капотом — стандартный стек (OCI) поверх тех же namespaces и cgroups.



изоляция и лимиты — запускает контейнер — управляет жизненным циклом — удобные команды

! Docker — это удобная обёртка над теми же namespaces и cgroups.

Что происходит при docker run — шаги в ядре

Docker не делает ничего волшебного — он по очереди дёргает системные вызовы ядра. Вот что происходит между вашей командой и работающим контейнером.

runc

```
# runc — упрощённо
mount(overlay) -> rootfs
clone(CLONE_NEWPID |
      CLONE_NEWNET | …)
write(memory.max, 512M)
pivot_root(rootfs)
exec("nginx")  # → PID 1
```

1. **Слои образа** → одна ФС «/» (overlay)
2. **clone()** → свои namespaces (PID, net, mnt…)
3. **cgroup** → лимиты CPU · RAM · I/O
4. **pivot_root** → корень = образ, хост скрыт
5. **exec** → команда становится PID 1 внутри

docker run: за каждым флагом — примитив ядра

Та же изоляция — теперь одной командой. Разберём типичный запуск: каждый флаг под капотом включает уже знакомый примитив.

`docker run`	`--memory 512m`	`-p 8080:80`	`nginx`
запустить	cgroup: лимит RAM	net ns: свой порт	образ → rootfs

терминал

```
$ docker run -d --memory 512m \
    -p 8080:80 --name web nginx
$ docker stats web  # виден лимит RAM
```

`--memory`, `--cpus` → cgroups

`-p`, `--network` → net namespace

`-v путь` → mnt namespace

Немного глупостей о контейнерах

Частые заблуждения, из-за которых потом «внезапно» падает прод.

«Контейнер — лёгкая ВМ»
На деле: это процесс на общем ядре; своей ОС и своего ядра нет

«Изоляция = безопасность»
На деле: ядро общее; это не граница безопасности, как у ВМ

«Образ пойдёт на любой ОС»
На деле: Linux-образу нужно Linux-ядро; на Mac/Windows — скрытая VM (помните про архитектуру!)

«Данные сохранятся сами»
На деле: r/w слой исчезает с контейнером — нужны volumes

«root внутри безопасен»
На деле: root в контейнере — тот же UID 0, от хоста его отделяют только ограничения прав

«Образ и контейнер — одно»
На деле: образ — шаблон (r/o), контейнер — запущенный экземпляр

ИТОГИ ЛЕКЦИИ

Что важно унести

1. **Контейнер — это процесс**
   изолированный процесс на общем ядре, а не мини-VM

2. **namespaces + cgroups**
   namespaces прячут ресурсы, cgroups их лимитируют

3. **VM тяжёлая, контейнер лёгкий**
   VM тащит гостевую ОС; контейнер стартует за миллисекунды

4. **Образ → контейнер**
   образ — read-only слои, контейнер добавляет r/w слой

Дальше в курсе — **Docker**: те же контейнеры на практике — Dockerfile и команды.


ВЫСШАЯ • ШКОЛА • ЭКОНОМИКИ •